Institute of Engineering and Technology (IET)

JK Lakshmipat University, Jaipur

Project Final Report

Threshold Cryptography for digital vaults

PREPARED BY

Aman Singh (2023BTECH006)
Ayush Choudhary (2023BTECH020)
Himanshu Gurjar (2023BTECH036)

FACULTY GUIDE

Dr. Surbhi Chhabra
Assistant Professor | Computer Science and Engineering

May 2026

Contents

1 Introduction

Digital vaults are systems that securely store and restrict access to sensitive information. Traditional vault systems depend on a *single secret key*, creating a single point of failure: lose the key, lose the data; one person gets the key, the vault is compromised.

Threshold Cryptography solves this problem. Instead of one key, it distributes cryptographic authority across multiple participants. Only when a *minimum threshold* of participants collaborate can the vault be opened.

1.1 Project Overview

This project implements a complete, enterprise-grade Threshold Digital Vault with the following cryptographic layers:

[colback=alertbg, colframe=accentcyan, title=System Architecture Layers]

Layer 1: AES-256-GCM — Authenticated symmetric encryption of the secret payload.

Layer 2: Shamir's Secret Sharing (SSS) — Threshold-based distribution of the AES Master Key.

Layer 3: Feldman Verifiable Secret Sharing (VSS) — Cryptographic share authentication using the Discrete Logarithm Problem.

1.2 Motivation

Real-world systems such as **Bitcoin hardware wallets**, **bank HSMs (Hardware Security Modules)**, and **corporate code-signing infrastructure** use the exact same layered approach. This project demonstrates a pedagogically complete implementation of these techniques.

2 Mathematical Foundations

2.1 Finite Fields

[Finite Field GF(p)] A Finite Field F_p (also written $GF(p)$) is a set of integers $\{0, 1, 2, \ldots, p-1\}$ where p is prime, equipped with addition and multiplication performed *modulo* p. Every non-zero element has a unique multiplicative inverse.

This project uses the **Mersenne Prime** $p = 2^{521} - 1$ as the field for Shamir's Secret Sharing. With 521 bits, this field is large enough to hold any standard secret without overflow.

2.2 Extended Euclidean Algorithm

To perform division in a finite field, we compute the **Modular Multiplicative Inverse**. Given k and prime p, we find x such that:

$$k \cdot x \equiv 1 \pmod{p}$$

The Extended Euclidean Algorithm finds integers x, y such that:

$$a \cdot x + b \cdot y = \gcd(a, b)$$

When $b = p$ and $\gcd(k, p) = 1$ (guaranteed for prime p), x is the modular inverse.

Listing 1: Extended GCD in Python

```
def extended_gcd(a, b):
    x0, x1, y0, y1 = 1, 0, 0, 1
    while b != 0:
        q, a, b = a // b, b, a % b
        x0, x1 = x1, x0 - q * x1
        y0, y1 = y1, y0 - q * y1
    return a, x0, y0   # gcd, x, y
```

2.3 Lagrange Interpolation over Finite Fields

Given t distinct points $(x_1, y_1), \ldots, (x_t, y_t)$ lying on a polynomial of degree $t - 1$, the unique polynomial can be reconstructed. Evaluating it at $x = 0$ recovers the secret constant term:

$$f(0) = \sum_{i=1}^{t} y_i \prod_{\substack{j=1 \\ j \neq i}}^{t} \frac{-x_j}{x_i - x_j} \pmod{p}$$

3 Shamir's Secret Sharing (SSS)

3.1 Overview

Shamir's Secret Sharing, proposed by Adi Shamir in 1979, is an **information-theoretically secure** (t, n) threshold scheme. The secret s is embedded as the constant term of a random polynomial of degree $t - 1$ over a finite field.

3.2 Key Generation — Polynomial Construction

[H] Shamir Secret Splitting [1] Secret integer s, threshold t, total shares n, prime p Generate random coefficients: $a_1, a_2, \ldots, a_{t-1} \xleftarrow{\$} F_p$ Define polynomial: $f(x) = s + a_1 x + a_2 x^2 + \cdots + a_{t-1} x^{t-1} \pmod{p}$ $i = 1$ to n Compute share: $(x_i, y_i) = (i,\ f(i) \bmod p)$ Compute fingerprint: $h_i = \text{SHA-256}(y_i)$ Distribute (x_i, y_i, h_i) to participant i

3.3 Security Property — Perfect Secrecy

[Information-Theoretic Security] For any $k < t$ shares, the posterior distribution of the secret s is identical to the prior (uniform over F_p). Formally:

$$H(s \mid x_{i_1}, x_{i_2}, \ldots, x_{i_k}) = H(s) \qquad \forall k < t$$

No computational assumption is needed — even a quantum computer gains zero advantage from fewer than t shares.

3.4 Python Implementation — Polynomial Evaluation

Listing 2: Horner's Method Polynomial Evaluation

```
def evaluate_polynomial(coefficients, x, prime):
    """Evaluates f(x) mod prime using Horner's method."""
    result = 0
    for coeff in reversed(coefficients):
        result = (result * x + coeff) % prime
    return result
```

4 Feldman Verifiable Secret Sharing (VSS)

4.1 The Problem with Basic SSS

In standard Shamir's Secret Sharing, a **malicious shareholder** (Director) can submit an altered y-value during recovery. The Lagrange Interpolation will then reconstruct the wrong AES key, permanently destroying the vault data. This is an active attack that basic SHA-256 checksums cannot prevent because:

1. Alice has share (x, y, h) where $h = \text{SHA-256}(y)$.
2. Alice creates a fake share (x, y', h') where $h' = \text{SHA-256}(y')$.
3. The checksum passes — both y' and h' are consistent with each other.

4. The vault is irreversibly destroyed.

4.2 Feldman's Solution — Cryptographic Commitments

Paul Feldman (1987) introduced a VSS scheme based on the **Discrete Logarithm Problem**. The dealer publishes a set of *commitments*:

$$C_j = g^{a_j} \pmod{p} \quad \text{for } j = 0, 1, \ldots, t-1$$

where g is a generator of a prime-order subgroup of Z_p^* and p is the **RFC 3526 Group 14 safe prime** (2048-bit).

4.3 Share Verification

Any participant can verify their share (x, y) by checking:

$$g^y \equiv \prod_{j=0}^{t-1} C_j^{x^j} \pmod{p}$$

Proof of correctness:

$$\prod_{j=0}^{t-1} C_j^{x^j} = \prod_{j=0}^{t-1} (g^{a_j})^{x^j} = g^{\sum_{j=0}^{t-1} a_j x^j} = g^{f(x)} = g^y \pmod{p}$$

4.4 Why an Attacker Cannot Forge a Share

[colback=alertbg, colframe=darkblue, title=**Security Argument**] To forge a share (x, y') that passes the Feldman check, an attacker must find y' such that $g^{y'} \equiv \prod C_j^{x^j} \pmod{p}$. This requires computing y' from a known power of g in a 2048-bit discrete logarithm group. The best known classical algorithm (GNFS) for this would require approximately 2^{112} operations — computationally infeasible for any foreseeable technology.

Listing 3: Feldman Share Verification

```
def feldman_verify_share(x, y, commitments, g, p, q):
    lhs = pow(g, y % q, p)          # g^y mod p
    rhs = 1
    x_power = 1
    for C_j in commitments:
        rhs = (rhs * pow(C_j, x_power, p)) % p
        x_power *= x                # x^j
    return lhs == rhs               # True = valid, False = tampered
```

5 Hybrid Encryption Architecture

5.1 Design Rationale

Shamir's Secret Sharing over a finite field limits the secret size to $< p$ (i.e., about 65 bytes for a 521-bit prime). Real-world vaults must store documents, certificates, and large files. The solution is **Hybrid Encryption**:

1. Encrypt the *data* with a fast symmetric key (AES-256).
2. Split the *symmetric key* using Shamir's Secret Sharing.

5.2 Complete Encryption Flow

[node distance=0.6cm and 1.2cm, box/.style=rectangle, rounded corners=5pt, draw=darkblue, fill=alertbg, text width=3.2cm, align=center, minimum height=1cm, font=, arrow/.style=-Latex[length=3mm], thick, color=accentcyan] [box] (secret) Secret Data (any size); [box, right=of secret] (aes) AES-256-GCM Encryption; [box, right=of aes] (vault) vault_data.json (ciphertext + nonce); [box, below=of aes] (key) Random AES-256 Key (32 bytes); [box, below=of key] (sss) Shamir's SSS + Feldman VSS; [box, left=of sss] (shares) Share Files director_1.json ... director_n.json; [arrow] (secret) – (aes); [arrow] (aes) – (vault); [arrow] (key) – (aes); [arrow] (key) – (sss); [arrow] (sss) – (shares);

Figure 1: Vault Creation: Hybrid Encryption + Threshold Key Distribution

5.3 AES-256-GCM

AES-256-GCM (Galois/Counter Mode) provides:

- **Confidentiality:** The data is encrypted under a 256-bit key. An exhaustive brute-force search over 2^{256} keys is computationally infeasible.
- **Integrity:** GCM appends a 128-bit authentication tag. Any modification to the ciphertext or nonce causes decryption to fail with an authentication error.
- **Freshness:** A random 96-bit nonce ensures that encrypting the same data twice produces completely different ciphertext.

Listing 4: AES-256-GCM Encryption and Decryption

```
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

# Encryption
aes_key = secrets.token_bytes(32)   # 256-bit key
nonce   = secrets.token_bytes(12)   # 96-bit nonce
aesgcm  = AESGCM(aes_key)
ciphertext = aesgcm.encrypt(nonce, plaintext.encode(), None)

# Decryption (fails if tampered)
plaintext = aesgcm.decrypt(nonce, ciphertext, None)
```

6 Security Analysis

6.1 Attack Scenario 1: Attacker Steals vault_data.json

What attacker has	ciphertext, nonce, Feldman commitments
What attacker needs	The 256-bit AES Master Key
Why it fails	AES-256 has 2^{256} possible keys. Brute-forcing at 10^{18} guesses/sec takes $\approx 10^{59}$ years.

Table 1: Attack Scenario 1 Analysis

6.2 Attack Scenario 2: Attacker Gets $t-1$ Share Files

What attacker has	$t-1$ valid share files
What attacker needs	1 more share
Why it fails	SSS provides **perfect secrecy**. The missing share is uniformly distributed over all 2^{256} field elements. No algorithm — classical or quantum — can narrow down the possibilities.

Table 2: Attack Scenario 2 Analysis

6.3 Attack Scenario 3: Malicious Director Forges a Share

What attacker does	Submits a tampered (x, y') share
Why it fails	Feldman VSS verification checks $g^{y'} \equiv \prod C_j^{x^j} \pmod{p}$. Without solving the 2048-bit Discrete Logarithm Problem, no y' exists that satisfies this equation.

Table 3: Attack Scenario 3 Analysis

6.4 Comparison with Basic Approaches

Feature	Password Only	Basic SSS	This Project
Single Point of Failure	Yes	No	No
Any-Size Data	Yes	No	Yes
Share Verification	N/A	Weak (SHA-256)	Feldman VSS
Quantum Resistant (key)	No	Yes	Yes (SSS layer)
Tamper Detection	No	Forgeable	Unforgeable

Table 4: Security Comparison

7 System Implementation

7.1 Project Structure

```
cp/cp/
          shamir_secret_sharing.py    # Finite Fields, SSS, Feldman VSS
          vault_converter.py          # String <-> Integer encoding
          advanced_digital_vault.py   # HybridDigitalVault class + CLI
          app.py                      # Flask Web Dashboard API
          templates/
                index.html              # Dark-themed web dashboard
          shares/                     # Generated director key files
          vault_data.json             # Encrypted vault storage
```

7.2 Key Components

Module	Responsibility
`shamir_secret_sharing.py`	Extended GCD, modular inverse, polynomial generation/evaluation, Shamir splitting, Feldman commitments, Lagrange interpolation.
`vault_converter.py`	Lossless UTF-8 string $\leftrightarrow$ large integer mapping via hexadecimal encoding.
`advanced_digital_vault.py`	`HybridDigitalVault`: orchestrates AES-256-GCM + Feldman SSS for vault creation and recovery. File I/O for shares and vault data.
`app.py`	Flask REST API: `/api/create-vault`, `/api/open-vault`, `/api/verify-share` endpoints.
`templates/index.html`	Single-page web dashboard with dark glassmorphism UI, drag-and-drop share upload, and real-time feedback.

Table 5: Module Responsibilities

7.3 How to Run

```
# Install dependencies
pip install flask cryptography

# Start the web dashboard
python app.py
# Open: http://127.0.0.1:5000

# Or run the CLI
python advanced_digital_vault.py
```

8 Web Dashboard

The project includes a modern, dark-themed web dashboard providing:

- **Create Vault Panel:** Enter secret data, configure (t, n) parameters, click to generate the encrypted vault and download individual JSON key share files.

- **Open Vault Panel:** Drag-and-drop JSON share files from multiple directors; the system verifies each share with Feldman VSS before decrypting.
- **Feldman Commitment Display:** Publicly shows the cryptographic proofs for educational transparency.
- **Real-time Status Feedback:** Success, error, and security alert messages displayed inline.

9 Results and Testing

All components were tested end-to-end. Below summarizes the verified behaviors:

Test Case	Expected	Result
Create vault with (3,5) scheme	5 shares, 1 vault file	**PASS**
Recover with exactly 3 shares	Secret decrypted	**PASS**
Recover with only 2 shares	Failure / wrong key	**PASS**
Submit tampered share (basic SSS)	SecurityAlert raised	**PASS**
Submit tampered share (Feldman VSS)	SecurityAlert raised	**PASS**
Modify vault_data.json ciphertext	AES authentication fails	**PASS**
Large secret ($>$ 65 chars)	AES encrypts successfully	**PASS**
Feldman verification (all shares)	All 5/5 marked VALID	**PASS**

Table 6: Test Results Summary

10 Conclusion

This project successfully demonstrates a complete, enterprise-grade implementation of **Threshold Cryptography for Digital Vaults**. By layering AES-256-GCM, Shamir's Secret Sharing, and Feldman Verifiable Secret Sharing, the system achieves:

- **No single point of failure** — No single person holds enough information to unlock the vault alone.
- **Perfect secrecy** — Fewer than t shares give an attacker zero mathematical information about the secret key.
- **Cryptographic integrity** — Feldman VSS ensures no participant can sabotage the vault with a forged share.
- **Unlimited payload size** — Hybrid AES encryption removes the field-size limitation of raw SSS.

- **Practical usability** — A modern web dashboard replaces error-prone manual copy-paste of 150-digit numbers.

The techniques demonstrated are directly applicable to real-world systems including cryptocurrency multi-signature wallets, corporate certificate authorities, and banking Hardware Security Modules (HSMs).

References

[1] A. Shamir, "How to Share a Secret," *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.

[2] P. Feldman, "A Practical Scheme for Non-Interactive Verifiable Secret Sharing," *28th Annual Symposium on Foundations of Computer Science (FOCS)*, 1987.

[3] T. Kivinen and M. Kojo, "More Modular Exponential (MODP) Diffie-Hellman groups for Internet Key Exchange (IKE)," *RFC 3526*, IETF, 2003.

[4] NIST, "Advanced Encryption Standard (AES)," *FIPS PUB 197*, November 2001.

[5] D. Boneh and V. Shoup, *A Graduate Course in Applied Cryptography*, 2023. Available: https://toc.cryptobook.us/